
Universidad de Guayaquil
Facultad de Ciencias Matemáticas y Físicas
Carrera de Software

Informe del Sprint 1

Ambiente reproducible, repositorio y DevOps

Prototipo web de inteligencia de negocios asistido por IA

Autoras	Emily Dayan Robles Alvarado y Lesly Samay Velásquez Anchundia
Identificador	SPR-01
Fecha de ejecución	24 de agosto al 2 de septiembre de 2026
Repositorio	LFXAS/prototipo-bi-ia
Línea base	Cierre aprobado e integrado mediante PR #15 y PR #16
Estado	Completado

Índice

1. Resumen ejecutivo	1
2. Ficha del sprint	1
3. Relación con el anteproyecto	1
4. Backlog ejecutado	2
5. Arquitectura entregada	3
6. Procedimiento ejecutado paso a paso	3
6.1. Revisión y congelación del alcance	3
6.2. Definición del enfoque Docker-first	3
6.3. Creación del backend	3
6.4. Creación del frontend	4
6.5. Preparación de PostgreSQL	4
6.6. Preparación de SQL Server y AdventureWorks	4
6.7. Orquestación aislada	4
6.8. Automatización de pruebas y operación	4
6.9. Inicialización de Git y publicación reproducible	4
6.10. Integración y entrega continuas	5
6.11. Documentación y trazabilidad	5
7. Trazabilidad técnica ampliada del incremento	6
8. Detalle del ciclo DevOps aplicado	6
8.1. Planificación y especificación	6
8.2. Construcción y configuración	7
8.3. Integración continua	7
8.4. Entrega continua	7
8.5. Operación y retroalimentación	7
9. Archivos producidos y responsabilidad	7
10. Validación manual equivalente	8
11. Cierre real del Sprint 1	8

12.Evidencias del incremento	8
12.1. Interfaz técnica	8
12.2. Contrato de la API	9
12.3. Estado de los servicios	10
12.4. Pruebas y entrega continua	11
13.Incidencias y resolución	11
14.Revisión del sprint	12
14.1. Criterios de aceptación	12
14.2. Incremento demostrable	12
15.Retrospectiva	12
16.Trabajo pendiente y propuesta para Sprint 2	12
17.Conclusión	13
17.1. Nota de evolución posterior	13
A. Comandos mínimos de demostración	13
B. Artefactos principales	13

1. Resumen ejecutivo

El trabajo realizado puede registrarse como el **Sprint 1** del proyecto. Produjo un incremento técnico terminado, verificable y reutilizable: el repositorio público con escritura controlada, el entorno Docker autocontenido, la aplicación base React/FastAPI, PostgreSQL, SQL Server con AdventureWorks, automatización de pruebas, CI/CD, publicación de imágenes y documentación reproducible.

El objetivo deliberado de este sprint fue reducir el riesgo tecnológico antes de desarrollar funcionalidades de negocio. Por ello, no se implementaron todavía autenticación, ETL, KPIs, copiloto con IA ni pronóstico. El módulo de seguridad RBAC quedó previsto mediante límites modulares y reglas arquitectónicas, tal como exige el alcance académico.

La denominación Sprint 1 es válida porque existieron un objetivo acotado, un conjunto priorizado de elementos de trabajo, criterios de aceptación, un incremento ejecutable, revisión mediante evidencias y una retrospectiva. Este informe formaliza dichas actividades para su incorporación en la tesis; no pretende afirmar ceremonias o métricas que no se hayan observado.

2. Ficha del sprint

Campo	Definición
Nombre	Base reproducible y canal DevOps
Objetivo	Dejar el entorno completo ejecutándose desde cero con Docker y publicarlo de forma controlada y repetible.
Resultado esperado	Una persona autorizada puede clonar el repositorio, configurar secretos locales y levantar el mismo ambiente sin instalar Python, Node.js ni motores de base de datos en el host.
Alcance funcional	Salud técnica y pantalla de estado; sin módulos de negocio.
Fuente externa	SQL Server con AdventureWorks 2022, accesible mediante un usuario de sólo lectura.
Datos internos	PostgreSQL con esquemas iniciales app y mart .
Automatización	Docker Compose, Make, scripts, GitHub Actions y GitHub Container Registry.
Definición de terminado	Construcción limpia, pruebas aprobadas, cuatro servicios saludables, restauración verificable, documentación compilada y CI/CD exitosa.

3. Relación con el anteproyecto

El anteproyecto corregido plantea una prueba de concepto con una fuente SQL Server/AdventureWorks, generación asistida por IA, aprobación humana, ETL hacia PostgreSQL, cinco KPIs, tres visualizaciones, insights explicables y un pronóstico mensual por regresión lineal evaluado con MAPE y RMSE.

El Sprint 1 no adelanta estas funciones. Establece la plataforma necesaria para implementarlas con trazabilidad y separación de responsabilidades. También incorpora el requerimiento institucional de un módulo de seguridad, conservándolo como arquitectura pendiente y no como funcionalidad simulada.

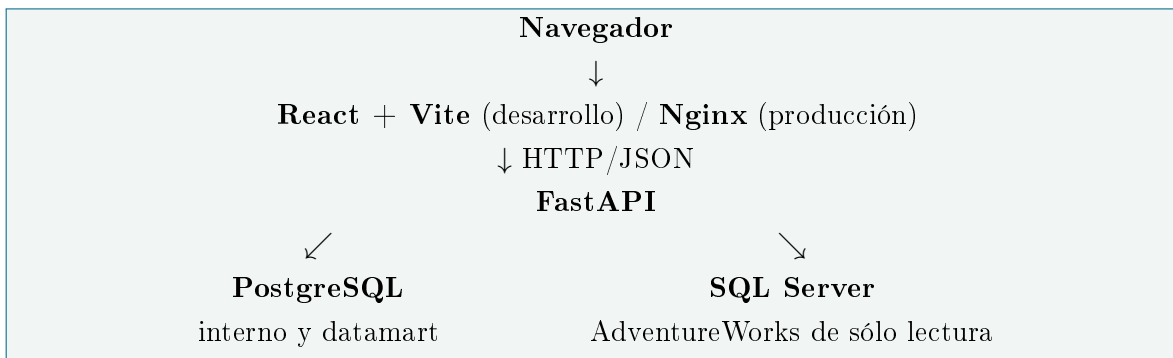
Necesidad del proyecto	Aporte del Sprint 1	Estado
Fuente SQL Server	Contenedor aislado y restauración automática de AdventureWorks 2022	Completado
Lectura controlada	Usuario <code>bi_reader</code> , intención de lectura y denegación de escritura	Completado
Datamart PostgreSQL	Instancia propia y esquemas <code>app/mart</code>	Completado
Aplicación web	React + TypeScript y API FastAPI conectados	Completado
Seguridad RBAC	Paquete, entidades previstas y reglas de autorización documentadas	Siguiente sprint
Módulos BI e IA	Límites de módulos definidos, sin lógica funcional	Siguiente sprint
Reproducibilidad	Docker, scripts, guía, imágenes y controles automáticos	Completado

4. Backlog ejecutado

ID	Elemento	Criterio de aceptación	Estado
SP1-01	Revisar el anteproyecto	Alcance incluido, exclusiones y límite de la fase documentados.	Hecho
SP1-02	Crear estructura del repositorio	Frontend, backend, infraestructura, documentación y automatización separados.	Hecho
SP1-03	Dockerizar FastAPI	Imagen de desarrollo, prueba y producción; endpoint de salud accesible.	Hecho
SP1-04	Dockerizar React/Vite	Recarga en desarrollo, prueba automatizada y Nginx en producción.	Hecho
SP1-05	Aislar PostgreSQL	Volumen propio, salud y esquemas iniciales sin tocar otras instancias.	Hecho
SP1-06	Aislar SQL Server	Volumen propio, restauración idempotente de AdventureWorks y usuario lector.	Hecho
SP1-07	Orquestar el conjunto	<code>docker compose up -build -d</code> inicia los cuatro servicios.	Hecho
SP1-08	Automatizar operación	<code>make bootstrap</code> , pruebas, validación y diagnóstico disponibles.	Hecho
SP1-09	Configurar Git/GitHub	Rama <code>main</code> , remoto privado y convenciones de contribución.	Hecho
SP1-10	Implementar CI/CD	Pruebas en cada cambio y publicación de cinco imágenes en GHCR.	Hecho
SP1-11	Documentar réplica	Otra máquina puede levantar por código o imágenes publicadas.	Hecho

ID	Elemento	Criterio de aceptación	Estado
SP1-12	Mantener trazabilidad	Bitácora LaTeX/PDF versionada y compilada automáticamente.	Hecho

5. Arquitectura entregada



Los servicios comparten únicamente la red creada por Compose. Los puertos de PostgreSQL y SQL Server se publican como 55432 y 51433, respectivamente, para evitar colisiones con las instancias habituales 5432 y 1433 del equipo anfitrión. Los volúmenes pertenecen exclusivamente al proyecto `bi-ia-prototype`.

FastAPI organiza los límites futuros en `security`, `parameters`, `metadata`, `copilot`, `etl`, `analytics`, `forecasting` y `reports`. Sólo el módulo `system` contiene comportamiento durante este sprint.

6. Procedimiento ejecutado paso a paso

6.1. Revisión y congelación del alcance

Se inspeccionaron las once páginas del anteproyecto. Se separaron las capacidades finales del prototipo de las tareas permitidas en la primera fase. El resultado se registró en `docs/scope.md`; la decisión principal fue no implementar módulos de negocio mientras el entorno no estuviera reproducible.

6.2. Definición del enfoque Docker-first

Se adoptó Docker como única dependencia de ejecución del equipo anfitrión. La decisión quedó registrada en `docs/decisions/0001-docker-first.md`. Cada componente propio posee un Dockerfile con una etapa acorde a desarrollo, pruebas o producción.

6.3. Creación del backend

Se creó una aplicación FastAPI con configuración mediante variables de entorno, enrutamiento versionado, conexión inicial a PostgreSQL, soporte de Alembic y pruebas. Se añadieron dos controles:

- `/api/v1/health/live`: confirma que el proceso de la API responde;

- `/api/v1/health/ready`: confirma además la conexión con PostgreSQL.

6.4. Creación del frontend

Se creó React con Vite y TypeScript. La pantalla técnica consulta el backend y presenta el estado de la API, los componentes disponibles y los módulos deliberadamente pendientes. En producción, Nginx sirve los archivos estáticos y redirige `/api/` al backend.

6.5. Preparación de PostgreSQL

La imagen propia parte de PostgreSQL y ejecuta un script de inicialización que crea los esquemas `app` y `mart`. Las credenciales se inyectan desde `.env`; el archivo real está excluido de Git y sólo se publica `.env.example`.

6.6. Preparación de SQL Server y AdventureWorks

Se creó una imagen que arranca SQL Server, descarga el respaldo oficial `AdventureWorks2022.bak` cuando el volumen está vacío, restaura la base de forma idempotente y crea `bi_reader`. La cuenta recibe capacidad de conexión y lectura, junto con denegaciones explícitas de escritura. La sonda de salud utiliza esa cuenta, no la cuenta administrativa.

6.7. Orquestación aislada

`compose.yaml` define los cuatro servicios, dependencias condicionadas por salud, red, puertos y volúmenes propios. El conjunto no referencia nombres ni volúmenes de contenedores preexistentes en el computador.

```
cp .env.example .env
# Reemplazar cada valor ChangeMe_* antes de continuar.
make bootstrap
```

6.8. Automatización de pruebas y operación

Se añadieron Makefile, `scripts/bootstrap.sh`, `scripts/wait-for-stack.sh` y `scripts/verify-environment.sh`. Estos recursos construyen, esperan la restauración, comprueban salud y ejecutan controles de calidad sin instalar herramientas de desarrollo en el host.

6.9. Inicialización de Git y publicación reproducible

Se inicializó Git con `main`, se aplicó una política para excluir secretos y datos locales, y se creó el repositorio [LFXAS/prototipo-bi-ia](https://github.com/LFXAS/prototipo-bi-ia). Su visibilidad vigente es pública para facilitar la réplica académica, mientras que la escritura permanece restringida a colaboradoras autorizadas. Las contribuciones siguen Conventional Commits y una plantilla de pull request.

6.10. Integración y entrega continuas

CI valida Compose y construye las etapas de prueba de frontend, backend e infraestructura. CD vuelve a comprobar la entrega y publica cinco imágenes públicas en GHCR: frontend, backend, PostgreSQL, SQL Server y documentación. Las etiquetas incluyen rama, versión y SHA. CD publica imágenes, pero todavía no despliega automáticamente una VM Azure.

Los volúmenes no se publican. El estado se reconstruye desde scripts, migraciones y el respaldo oficial; esta decisión evita distribuir contraseñas o datos dependientes de una máquina.

6.11. Documentación y trazabilidad

Se crearon README, guía de réplica, descripción de arquitectura, ADR y bitácora LaTeX. El repositorio incluye reglas para que futuras sesiones de desarrollo actualicen pruebas y documentos antes de entregar cambios. Como cierre, se adoptó SDD para los sprints posteriores: una especificación versionada definirá cada capacidad funcional antes de su implementación.

7. Trazabilidad técnica ampliada del incremento

El incremento no se limita a que la pantalla responda. La trazabilidad conecta una necesidad del Sprint 1 con un archivo, un control automático y una evidencia observable.

Necesidad	Implementación	Control	Evidencia
Repetibilidad	Dockerfiles, Compose y plantillas de entorno	<code>make compose-check</code>	Configuraciones interpretadas sin error
Aislamiento	Nombre Compose, puertos y volúmenes propios	<code>docker compose ps</code>	Recursos prefijados por el proyecto
Calidad backend	Etapas Docker <code>test</code>	Ruff, formato, Mypy y Pytest	Construcción exitosa en CI
Calidad frontend	Etapas Docker <code>test</code>	ESLint, Vitest y Vite build	Construcción exitosa en CI
Fuente lectora	Usuario <code>bi_reader</code> y sonda real	Consulta permitida y escritura denegada	SQL Server saludable
Persistencia	Volúmenes nombrados	Reinicio y recreación sin <code>-volumes</code>	Datos conservados
Integración	PR hacia <code>develop</code>	Aprobación y controles obligatorios	PR #15 fusionado
Promoción	PR <code>develop</code> hacia <code>main</code>	Política de flujo y CI	PR #16 fusionado
Entrega	Workflow CD y GHCR	Verificación previa y matriz	Cinco imágenes publicadas
Documentación	LaTeX dentro de Docker	Compilación y <code>git diff -exit-code</code>	Tres PDF versionados
Réplica	Compose de desarrollo y de entrega	Prueba en VM Ubuntu	Aplicación levantada fuera del equipo principal
Gobierno futuro	SDD, plantilla y especificaciones	Revisión antes del código	Contrato de Sprint 2 preparado

8. Detalle del ciclo DevOps aplicado

8.1. Planificación y especificación

El alcance se obtuvo del anteproyecto y se congeló en `docs/scope.md`. La decisión Docker-first se formalizó en un ADR. El Sprint 1 se ejecutó antes de adoptar formalmente SDD; como mejora de proceso, el cierre incorporó `docs/sdd-workflow.md`, una plantilla y la especificación preliminar del Sprint 2.

8.2. Construcción y configuración

`.env.example` documenta los valores de desarrollo sin secretos reales. `compose.yaml` construye servicios desde el código. `compose.prod.yaml` selecciona etapas estables durante una construcción local. `compose.release.yaml` descarga las imágenes publicadas y usa otros puertos y volúmenes. Esta separación permitió mantener simultáneamente un entorno editable y otro de comprobación.

8.3. Integración continua

El workflow `ci.yaml` se activa en pushes y PR hacia `develop` o `main`. Sus trabajos validan el flujo de ramas, las variantes Compose, las etapas de prueba de FastAPI y React, las imágenes de infraestructura y los tres documentos. Los ejecutores son efímeros: al terminar no dejan servicios de desarrollo activos.

La protección de ramas agrega una aprobación humana a la verificación automática. CI puede detectar incumplimientos, pero no decide si el cambio satisface el objetivo académico; esa responsabilidad permanece en la revisión.

8.4. Entrega continua

El workflow `cd.yaml` se activa al integrar en `main` o crear una etiqueta `v*`. Primero revalida backend, frontend y la configuración de entrega. Después una matriz construye y publica cinco imágenes en GHCR. Cada artefacto recibe una etiqueta de rama o versión y otra derivada del SHA del commit. El CD entregado no despliega Azure; ese alcance se mantiene explícitamente pendiente.

8.5. Operación y retroalimentación

Las sondas de salud, registros de Docker, estado de Actions, digest de imágenes y prueba de réplica forman el ciclo de retroalimentación. Los incidentes encontrados se transformaron en cambios versionados: corrección de salud IPv4, espera de recuperación de AdventureWorks, permisos efectivos del lector, caché LaTeX escribible y exclusión correcta de secretos.

9. Archivos producidos y responsabilidad

Archivo o carpeta	Responsabilidad dentro del Sprint 1
<code>backend/Dockerfile</code>	Ambientes Python de desarrollo, prueba y producción; controlador ODBC y usuario no privilegiado.
<code>frontend/Dockerfile</code>	Dependencias reproducibles, servidor Vite, controles y entrega Nginx.
<code>infra/postgres/</code>	Imagen PostgreSQL e inicialización de los esquemas <code>app</code> y <code>mart</code> .
<code>infra/sqlserver/</code>	SQL Server, restauración idempotente y cuenta de sólo lectura.
<code>infra/docs/</code>	Compilador LaTeX versionado como imagen.
<code>compose.yaml</code>	Desarrollo con recarga, bases locales y perfil de vista Nginx.

Archivo o carpeta	Responsabilidad dentro del Sprint 1
compose.prod.yaml	Sobreescritura para etapas estables construidas localmente.
compose.release.yaml	Entrega aislada que consume GHCR sin compilar.
Makefile y scripts/	Interfaz operativa, esperas, verificación y política de ramas.
.github/workflows/ci.yml	Integración continua sin permiso de publicación.
.github/workflows/cd.yml	Verificación y publicación de cinco imágenes.
.github/dependabot.yml	Propuestas semanales de mantenimiento de dependencias.
docs/	Alcance, arquitectura, decisiones, réplica, SDD, manual, informe y bitácora.

10. Validación manual equivalente

La siguiente secuencia resume lo que el equipo ejecutó o automatizó para declarar terminado el incremento:

```
cp .env.example .env
# Sustituir marcadores ChangeMe_*
docker compose config --quiet
docker compose --profile delivery config --quiet
docker compose --env-file .env.release.example \
  -f compose.release.yaml config --quiet
docker build --target test -t backend:test backend
docker build --target test -t frontend:test frontend
make docs
make bootstrap
docker compose ps
curl --fail http://localhost:8000/api/v1/health/ready
```

11. Cierre real del Sprint 1

El PR #15 integró en **develop** el cierre documental y la adopción de SDD. El PR #16 promovió **develop** a **main** con aprobación, controles satisfactorios y sin conflictos. Por ello, el estado del incremento es cerrado y promocionado; Azure, RBAC, parámetros y capacidades BI permanecen fuera de este sprint.

12. Evidencias del incremento

12.1. Interfaz técnica

La Figura 1 muestra la pantalla ejecutándose en `localhost:5173`. La insignia verde confirma que el frontend pudo consultar la API. La interfaz declara de forma transparente qué módulos quedan para sprints posteriores.



Figura 1: Pantalla inicial del prototipo y estado del ambiente.

12.2. Contrato de la API

La documentación OpenAPI de la Figura 2 presenta únicamente los endpoints técnicos habilitados en esta fase. Esta evidencia confirma el cumplimiento del límite del sprint: infraestructura funcional sin módulos de negocio prematuros.

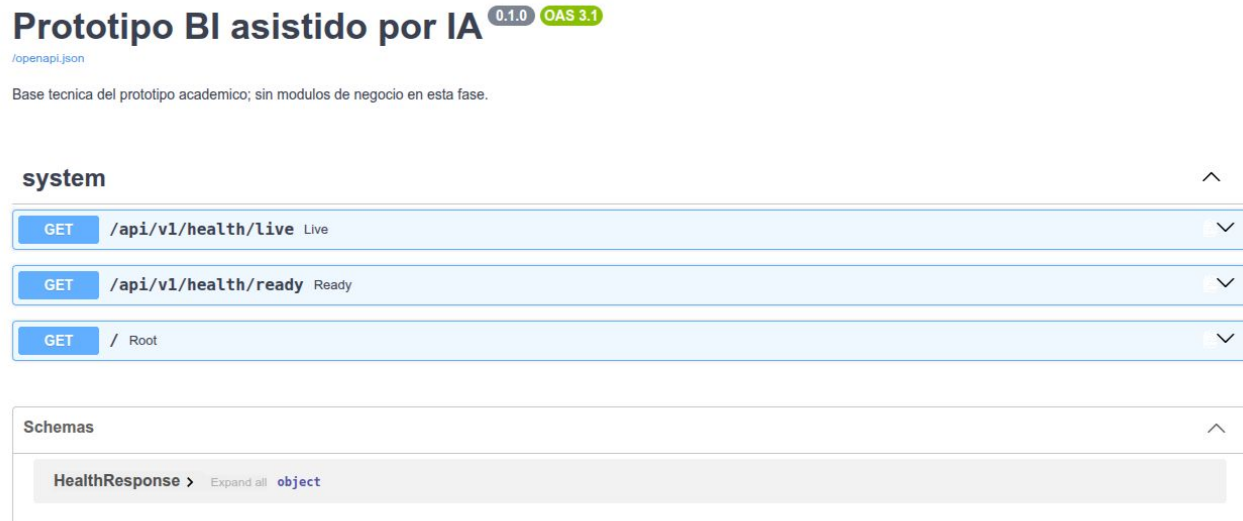


Figura 2: Documentación OpenAPI de FastAPI en localhost:8000/docs.

12.3. Estado de los servicios

Servicio	Función	Puerto anfitrión	Estado observado
frontend	React/Vite y proxy a la API	5173	Saludable
backend	FastAPI y OpenAPI	8000	Saludable
postgres	Datos internos y futuro datamart	55432	Saludable
sqlserver	AdventureWorks externo	51433	Saludable

La respuesta del control de preparación fue:

```
{"status":"ok","service":"Prototipo BI asistido por IA",
  "version":"0.1.0","postgres":"ok"}
```

AdventureWorks fue reconstruida desde un volumen vacío. La verificación encontró 71 tablas. La cuenta `bi_reader` pudo ejecutar `SELECT` y no pudo insertar, confirmando el control del motor y no sólo una opción de conexión.

12.4. Pruebas y entrega continua

Control	Resultado	Evidencia
Backend	Aprobado	Ruff, Mypy y dos pruebas Pytest en imagen Docker
Frontend	Aprobado	ESLint, Vitest y build Vite en imagen Docker
Compose	Aprobado	Variantes de desarrollo y publicación válidas
Bitácora	Aprobado	PDF reproducible sin diferencias en CI
CI	Éxito	Ejecución 32749237009
CD	Éxito	Ejecución 32749237059
GHCR	Publicado	Cinco imágenes públicas con etiquetas <code>main</code> y <code>sha-*</code>

13. Incidencias y resolución

Incidencia	Diagnóstico	Resolución
Caché npm sin escritura	La caché predeterminada estaba en una ubicación restringida.	Se generó el lockfile con una caché temporal controlada.
Formato/import del backend	Las reglas vigentes de Ruff detectaron diferencias.	Se corrigió el código antes de congelar la imagen de prueba.
Inicialización duplicada de pruebas	<code>jest-dom</code> se cargaba dos veces.	Se eliminó la importación redundante y Vitest pasó.
Vulnerabilidades de npm	Las versiones iniciales de Vite/Vitest tenían avisos conocidos.	Se actualizaron a versiones corregidas; el lockfile quedó sin vulnerabilidades reportadas.
Salud de frontend por IPv6	BusyBox resolvía <code>localhost</code> por IPv6 y Vite atendía en IPv4.	La sonda cambió a <code>127.0.0.1</code> .
Lector de SQL Server sin conexión	Una denegación amplia de <code>CONTROL</code> anuló un permiso subordinado.	Se retiró esa denegación y se mantuvieron denegaciones explícitas de escritura.
Advertencias de Node.js en Actions	Varias acciones utilizaban una generación anterior del runtime.	Se actualizaron las acciones oficiales y se repitieron CI/CD con éxito.
Invitaciones por correo pendientes	La API de colaboradores requiere nombres de usuario y los correos no devolvieron cuentas públicas.	Se aplazaron las invitaciones hasta disponer de los identificadores GitHub o una sesión web autenticada.

14. Revisión del sprint

14.1. Criterios de aceptación

Cumple. El entorno se reconstruyó después de eliminar exclusivamente sus volúmenes.

Cumple. Los cuatro servicios quedaron saludables y los endpoints respondieron.

Cumple. AdventureWorks se restauró y el lector no obtuvo escritura.

Cumple. Las pruebas del frontend y backend finalizaron correctamente.

Cumple. El repositorio público y las cinco imágenes públicas quedaron publicados sin secretos ni volúmenes.

Cumple. Existe una guía reproducible para el computador actual y para terceros autorizados.

Cumple. El RBAC está previsto y todavía no se presenta como implementado.

14.2. Incremento demostrable

El incremento puede demostrarse con una sola secuencia: clonar, crear `.env`, ejecutar `make bootstrap`, abrir el frontend y OpenAPI, y consultar los controles de salud. La variante `compose.release.yaml` permite repetir el entorno usando imágenes GHCR sin compilar localmente.

15. Retrospectiva

Categoría	Aprendizaje o acción
Mantener	Docker como frontera del entorno, pruebas dentro de imágenes y secretos fuera de Git.
Mantener	Validación real del usuario lector y no sólo <code>ApplicationIntent=ReadOnly</code> .
Mantener	Documentación y PDF como parte de la definición de terminado.
Mejorar	Congelar etiquetas de versión y registrar digest de cada imagen para entregas académicas.
Mejorar	Incorporar migraciones desde el primer cambio que añada tablas de aplicación.
Evitar	Permisos SQL demasiado amplios, dependencias sin lockfile y verificaciones ejecutadas fuera de la raíz.
Próxima acción	Aprobar la especificación SDD y luego implementar el núcleo RBAC y los parámetros mediante migraciones y pruebas.

16. Trabajo pendiente y propuesta para Sprint 2

El Sprint 1 termina sin deuda funcional oculta: los módulos de negocio siguen explícitamente pendientes. Se propone que el Sprint 2 tenga como objetivo *seguridad y parámetros mínimos* e incluya:

1. aprobar primero `docs/specs/SPR-02-rbac-y-parametros.md` como contrato del sprint;
2. migración inicial para usuarios, roles, permisos, asignaciones, menús y auditoría;

3. autenticación segura y autorización por permiso en FastAPI;
4. menú React derivado de permisos, sin sustituir la validación del backend;
5. parámetros de conexión aprobados y prueba de SQL Server en modo lector;
6. pruebas de denegación por defecto, migraciones y trazabilidad;
7. mantener la documentación, evidencia y especificación enlazadas en cada PR.

17. Conclusión

El Sprint 1 alcanzó su objetivo: existe una base técnica ejecutable, aislada, automatizada y documentada que reduce el riesgo de integración para el resto del prototipo. El resultado no es una simulación del producto final; es el incremento de infraestructura que permite desarrollar los módulos académicos posteriores sobre una línea base verificable.

Para efectos de la tesis, este sprint aporta evidencia de planificación incremental, decisiones arquitectónicas, gestión de riesgos, control de configuración, pruebas, integración continua, entrega continua y reproducibilidad del ambiente.

17.1. Nota de evolución posterior

El perfil Docker opcional de Ollama y su modelo local se incorporan posteriormente como extensión del Sprint 2 para probar la configuración LLM. No forman parte del alcance ni de la evidencia histórica del Sprint 1; reutilizan la base Docker, los volúmenes nombrados y las prácticas de configuración segura establecidas en este informe. Su operación y verificación se documentan en el manual técnico vigente.

A. Comandos mínimos de demostración

```
cd "/home/ceo/Proyectos/BI"
make bootstrap
docker compose ps
curl http://localhost:8000/api/v1/health/live
curl http://localhost:8000/api/v1/health/ready
make verify
```

B. Artefactos principales

- `compose.yaml` y `compose.release.yaml`.
- `backend/`, `frontend/` e `infra/`.
- `scripts/bootstrap.sh`, `scripts/wait-for-stack.sh` y `scripts/verify-environment.sh`.
- `.github/workflows/ci.yml` y `.github/workflows/cd.yml`.
- `docs/replication-guide.md`, `docs/architecture.md` y `docs/logbook/bitacora.tex`.

- `docs/sdd-workflow.md` y `docs/specs/`, como control de evolución desde el Sprint 2.
- `AGENTS.md`, que conserva las reglas para el desarrollo asistido y automático.